



IO管理与输入输出

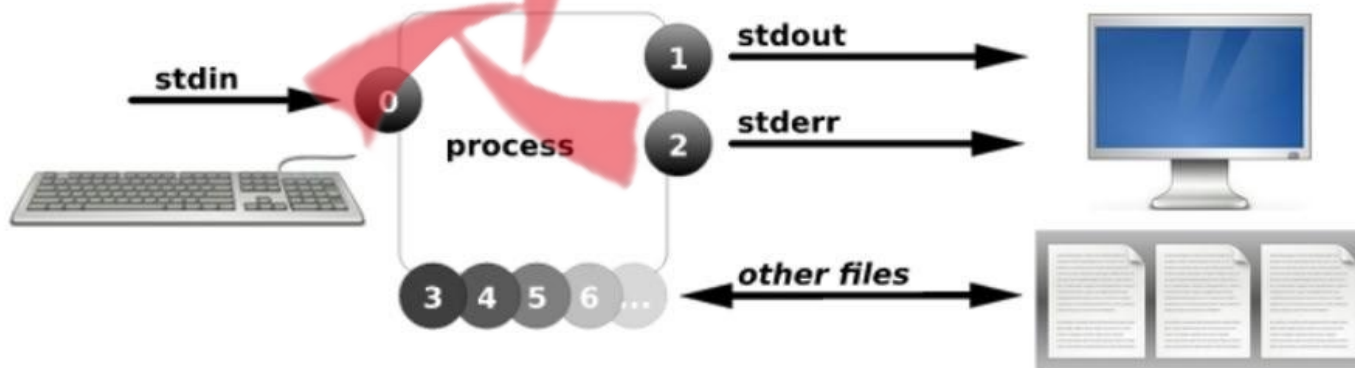
誉天教育

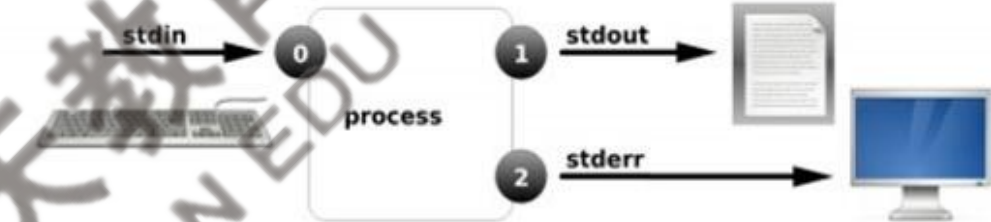
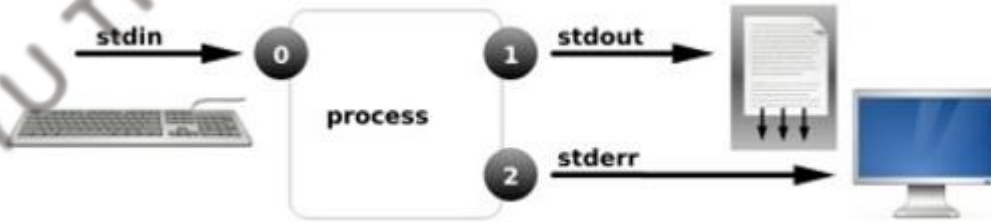
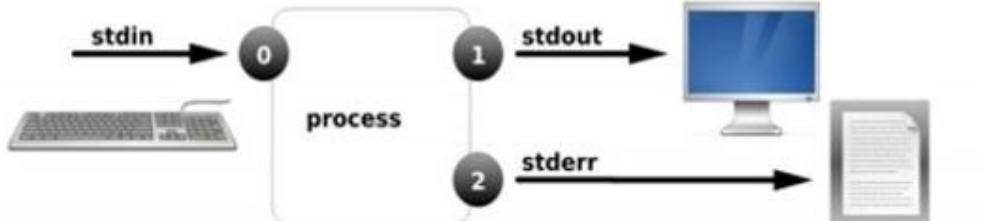
- 标准输入输出和错误
- 重定向
- 管道符
- 三通管道

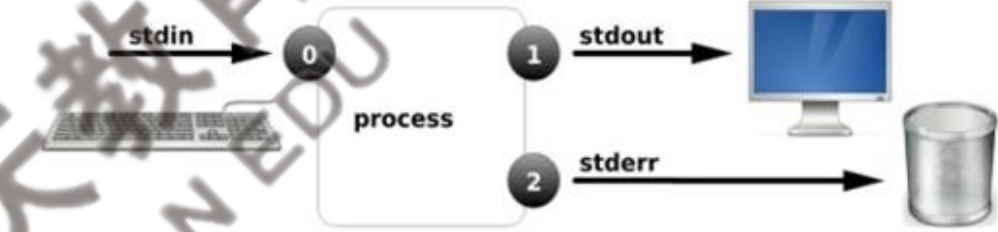
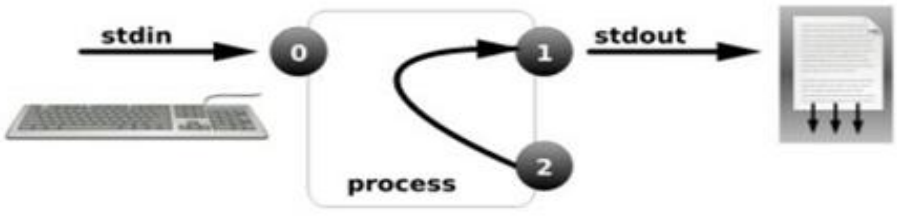


- 进程IO通道（文件描述符）

编号	通过名称	描述	默认设备	用法
0	stdin	标准输入	键盘	仅读取
1	stdout	标准输出	显示器	仅写入
2	stderr	标准错误输出	显示器	仅写入
3+	filename	其他文件	无	读取或者写入



用法	说明	视觉辅助
> file	重定向stdout以覆盖文件	 The diagram illustrates a process with three standard streams: stdin (0), stdout (1), and stderr (2). stdin is connected to a keyboard icon. stdout (1) is redirected to a file icon, with three downward arrows indicating that the file's previous content is being overwritten. stderr (2) is connected to a monitor icon.
>> file	重定向stdout以追加到文件	 The diagram illustrates a process with three standard streams: stdin (0), stdout (1), and stderr (2). stdin is connected to a keyboard icon. stdout (1) is redirected to a file icon, with three downward arrows indicating that new content is being appended to the end of the file. stderr (2) is connected to a monitor icon.
2> file	重定向stderr以覆盖文件	 The diagram illustrates a process with three standard streams: stdin (0), stdout (1), and stderr (2). stdin is connected to a keyboard icon. stdout (1) is connected to a monitor icon. stderr (2) is redirected to a file icon, with three downward arrows indicating that the file's previous content is being overwritten.

用法	说明	视觉辅助
<code>2>/dev/null</code>	重定向stderr到/dev/null	
<code>&>file</code> 等同于 <code>>file 2>&1</code>	重定向stdout和stderr以覆盖同一个文件	
<code>&>>file</code> 等同于 <code>>>file 2>&1</code>	重定向stdout和stderr以追加同一个文件	

示例-输出重定向

以下命令都是普通用户 **test** 执行

- 标准正确输出重定向 >

```
[test@rhel9 ~]$ find /etc/ -name passwd > aaa.txt
```

- 标准错误输出重定向 2>

```
[test@rhel9 ~]$ find /etc/ -name passwd 2> bbb.txt
```

- 标准正确错误输出重定向到/dev/null

```
[test@rhel9 ~]$ find /etc/ -name passwd &> bbb.txt
```

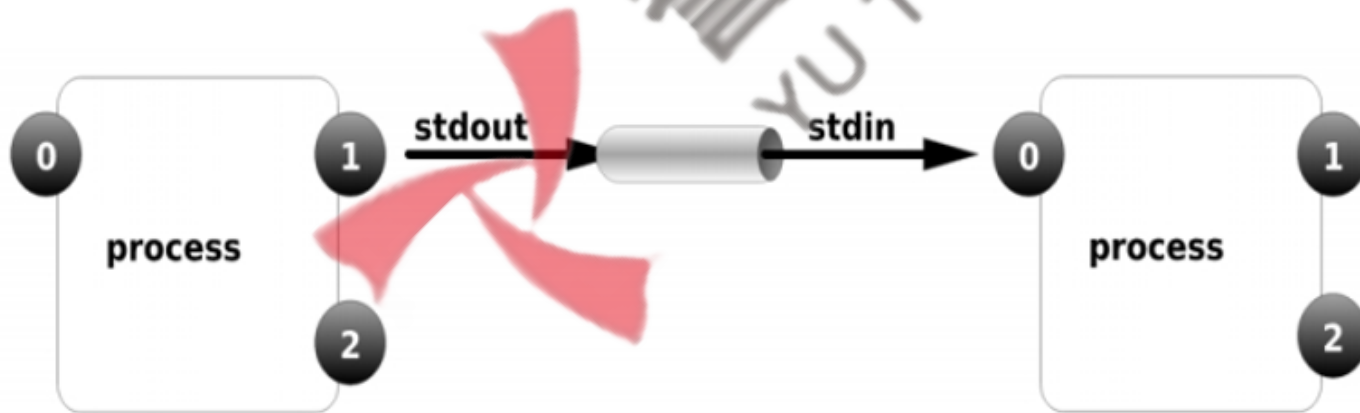


- 输入重定向 <
 - 从文件中接受数据到标准输入
- 示例：
- `wc -l < /etc/passwd`

```
[root@rhel9 ~]# wc -l < /etc/passwd
40
```

- `tr` 转换字符命令
- `$ tr `A-Z` `a-z` < .bash_profile`
- 以上命令将把bash_profile文件中的大写字符全部转换成小写字符

- 管道符 (|) 能够接多个命令
- 语法：
 - `command1 | command2 | command3`
- 将前面一条命令执行的结果输出作为管道后面命令的参数
- 错误的输出不能通过管道



示例-管道符传参

- less:一次查看一个页面的输入

```
$ ls -l /etc | less
```

- echo:打印后面的字符串

```
$ echo redhat | passwd --stdin root
```

- grep:过滤字符串

```
$ ls / | grep y
```

- wc -l:统计总行数

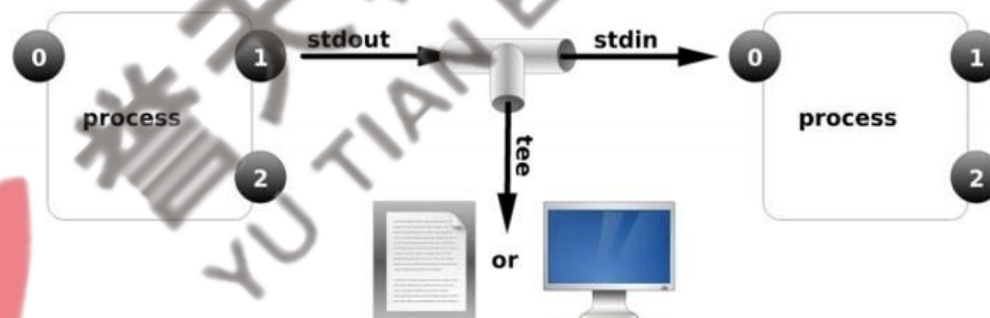
```
$ cat /etc/passwd | wc -l
```

- tee用法

`$ command1 | tee filename | command2`

- 存储command1的stdout在filename里，然后通过管道传给command2
- 使用：

- ◆ 故障诊断复杂的管道
- ◆ 同时查看和记载输出



示例：将列出的内容存到test.txt文件中，并且使用less查看页面输出

```
[root@rhel9 ~]# ls -l | tee test.txt | less
```

- 使用 <<**END** 从键盘重定向多行到STDIN
- 接收所有的标准输入直到输入**END**字符
- 例如：

```
$ mail -s "Please Call" jane@example.com <<END  
>HI Jane,  
>Please give me a call when you get in.We may need  
>to do some maintenance on server1  
>Details when you're on-site,  
>Boris  
>END
```

P：END可以是任意字符，用什么字符开始就什么字符结束

- ✓ 标准输入输出
- ✓ 重定向
- ✓ 管道符
- ✓ 多行输出

